Acknowledgement

I, Kasinath Shelly, would like to thank our Computer Science teacher, Thendral Kala Rajendran, for guidance during the planning and preparation of this project. I also thank my school and family for their support. Working on this project helped me practise Python functions, modular programming, graphical user interfaces, database concepts, and SQL connectivity.

Student name: Kasinath Shelly

Signature: _____

Project Synopsis

The Task Management System provides a desktop screen to add, search, filter, complete, and delete personal tasks. Each task can be assigned a priority and optional due date. Task records are kept in MySQL, and the program separates the Tkinter interface from database operations.

Project at a glance

Item	Details
Project title	Task Management System
Class / subject	Class XII Computer Science, Subject Code 083
Programming language	Python 3
Interface	Tkinter desktop interface
Database	task_manager_db in MySQL
Main learning areas	Functions, modules, input validation, SQL, Python–SQL connectivity

The CBSE Class XII Computer Science syllabus for 2026–27 includes database management and Python–SQL connectivity, and describes the project as an application of concepts learnt in Classes XI and XII. This report uses a conventional school project-file order; the CBSE syllabus does not prescribe one mandatory cover-to-appendix layout.

Index

Page numbers are generated automatically. Select a heading in the PDF outline to jump to that section.

Acknowledgement 1

Project Synopsis 2

1. Introduction 4

2. Requirements and Tools 4

3. System Design 4

4. Database Design 5

5. Modules and Implementation 6

6. Test Plan 7

7. Sample Application Output 7

8. Conclusion and Future Scope 8

9. Bibliography 8

10. Viva Voce Preparation 9

Appendix A. Complete Source Code 9

Appendix B. SQL Setup Script 21

1. Introduction

People often keep school work and personal reminders in different places. This application gives a student one small task list with a title, optional description, due date, and completion status. The list can be managed from a simple desktop window.

Problem statement

A student needs a clear way to record tasks, review work by priority or status, and remove or complete entries. The program stores each task in MySQL and displays the filtered records in a graphical list.

Objectives

- Create a clear desktop interface using Tkinter.
- Save and retrieve task records using MySQL.
- Use SQL INSERT, SELECT, UPDATE, and DELETE operations.
- Separate the user interface and database functions into modules.
- Validate required input and handle connection or query errors with messages.

Scope and limitations

The program is a single-user local task list. It supports task creation, display, completion, and deletion. It does not include user accounts, cloud synchronization, reminders, or multi-user permissions. A local MySQL Server and correct account settings are required.

2. Requirements and Tools

Type	Requirement
Hardware	A computer capable of running Python, a graphical display, and sufficient storage for MySQL.
Software	Python 3, MySQL Server, and a terminal or Python IDE.
Python package	mysql-connector-python.
Built-in library	Tkinter for the graphical interface.
Database setup	Run the supplied schema.sql once, then set local MySQL credentials in db_config.py.

The report demonstrates the program structure and user interface. Before a school demonstration, configure MySQL on the presentation computer, run the supplied schema, and enter the correct local login in the configuration file.

3. System Design

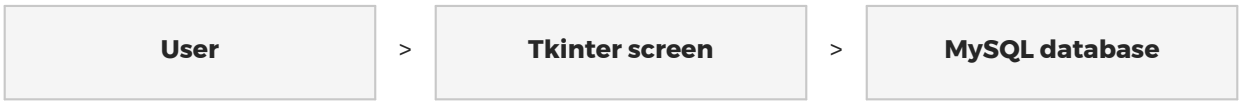
Main features

- Add a task title, optional details, due date, and priority.

- Search by task text and filter by pending or completed status.
- Display saved tasks with IDs, priority, and current status.
- Mark the selected task as completed.
- Filter tasks by search text or status.
- Delete a selected task after a confirmation prompt.
- Refresh the list from the MySQL database.

Program flow

The user works in the Tkinter screen. Button events call a database function. The database function sends a parameterized SQL operation to MySQL, and the refreshed records return to the table view.



Algorithm

1. Open a connection to the task_manager_db database and ensure that the tasks table exists.

2. Build the Tkinter window and load the saved task records.

3. Wait for a button action from the user.

4. For Add, check the title, then insert the task into MySQL.

5. For Mark Completed, update the selected task status.

6. For Delete, confirm the request and delete the selected task.

7. Refresh the table after a successful database change; repeat until the window is closed.

Input validation and error handling

A blank title is rejected. Database errors are displayed in a dialog. Task IDs are read from the selected table row. Due dates are entered in YYYY-MM-DD format; MySQL checks whether a supplied value is a valid date.

4. Database Design

The application stores its records in the MySQL database **task_manager_db**. The SQL file supplied with the project creates the database and its table. The program uses parameter placeholders for values passed to SQL commands.

Field	Type	Constraint / use
task_id	INT	Primary key, automatically assigned

Field	Type	Constraint / use
title	VARCHAR(100)	Required task title
description	VARCHAR(255)	Optional task details
due_date	DATE	Optional due date
priority	VARCHAR(10)	Low, Normal, or High importance
status	VARCHAR(15)	Pending or Completed

SQL operations used

Operation	Purpose in the project
INSERT	Add a task with its title, details, and optional due date.
SELECT	Fetch saved tasks and show them in the table.
UPDATE	Change a selected task status to Completed.
WHERE	Filter task rows by text and selected status.
DELETE	Remove a selected task by its ID.

Connectivity sequence

Python imports mysql.connector, opens a connection with the settings in db_config.py, creates a cursor, runs a parameterized SQL command, commits changes when needed, fetches records for display, and closes the connection.

5. Modules and Implementation

Module / file	Responsibility
app.py	Tkinter screen, button actions, input checks, and messages.
database.py	MySQL connection, table creation, and task queries.
db_config.py	Host, username, password, and database settings.
schema.sql	Database and tasks table setup.
requirements.txt	MySQL Connector/Python package name.
windows.bat	Checks Python startup, rebuilds a broken environment, installs dependencies, prepares MySQL, and launches the app.

The app.py class creates the window, entries, table, and buttons. Each button calls a method. The database.py module opens and closes a MySQL connection for each operation. Parameterized statements pass user values separately from SQL text.

6. Test Plan

The following cases are included as a test plan. Run them on the configured computer before signing or submitting this report; expected behavior is listed here, not represented as a completed local test.

Test	Input / action	Expected behavior
Empty title	Submit Add with no title.	A warning appears and no record is saved.
Add task	Enter a title and valid optional date.	A new Pending row appears in the table.
View / refresh	Click Refresh after adding a task.	The row is loaded from MySQL.
Search and filter	Enter text or choose a status.	Only matching tasks are displayed.
Complete task	Select a Pending row and click Mark completed.	Its status changes to Completed.
Delete task	Select a row, confirm deletion.	The row is removed from the table and database.
Database unavailable	Stop MySQL and start the program.	A connection error message explains what to check.

7. Sample Application Output

The image below is an illustrative UI preview populated with sample records. It shows the screen layout and expected output; it is not a live screenshot from a connected MySQL session. Replace it with a screenshot from your own running copy if your school requires direct evidence.

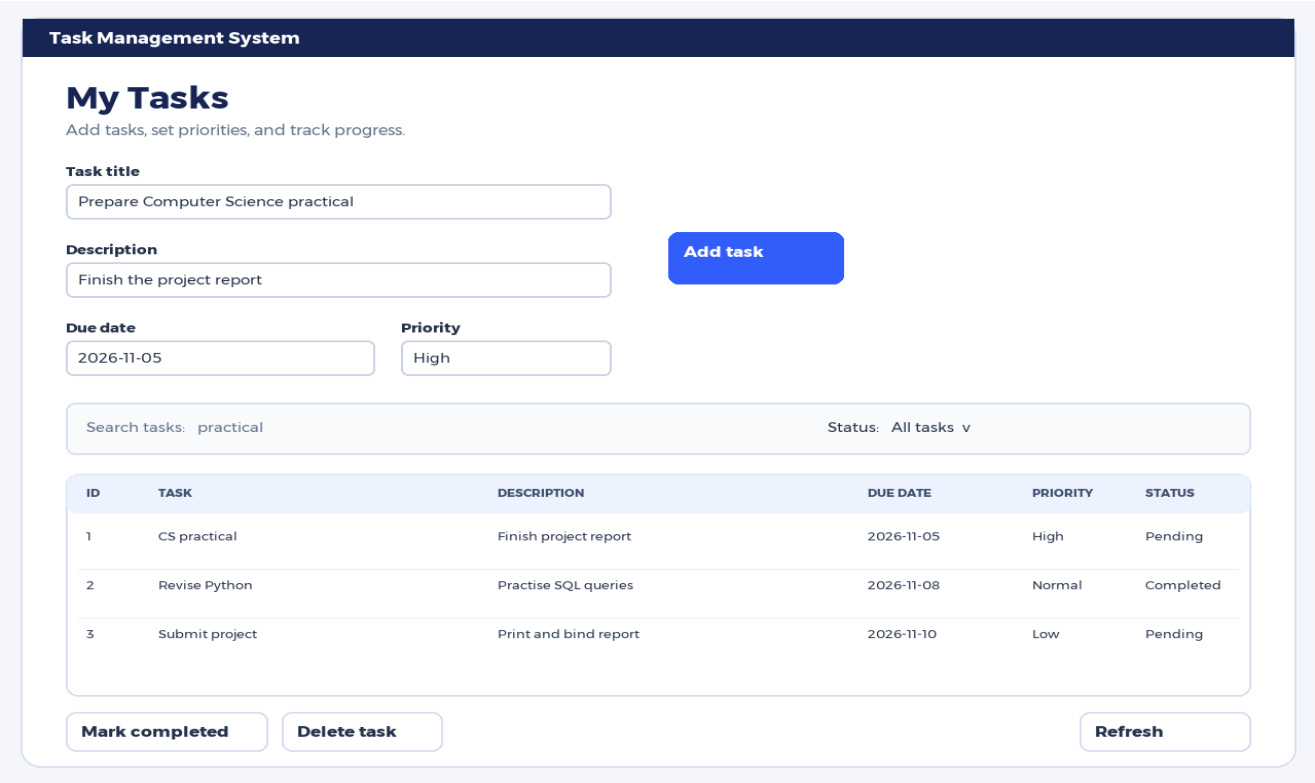


Figure 1. Illustrative Task Management System window with sample data.

How to run the program

Extract the project archive, run schema.sql in MySQL Workbench or from a MySQL command line, install the requirements, update db_config.py with the local MySQL login, then run mac.sh on macOS or windows.bat on Windows. The launcher installs the Python connector, applies the SQL file, saves the MySQL login for the app, and starts the program.

8. Conclusion and Future Scope

The project demonstrates a small practical use of Python, Tkinter, SQL, and a relational database. Its features let a student store tasks and change their status. Separating the screen and database code makes the project easier to review and maintain.

Possible improvements

- Add a reminder when a due date is approaching.
- Add task categories such as school, home, and activities.
- Allow the user to edit an existing task.

9. Bibliography

1. Central Board of Secondary Education, Computer Science, Subject Code 083, Class XII, Curriculum 2026–27, Practical and Project sections. Official CBSE curriculum PDF.

2. Python Software Foundation, Python 3 Documentation. docs.python.org/3.

3. Oracle, MySQL Connector/Python Developer Guide. dev.mysql.com/doc/connector-python.

4. Project source files and schema included in the accompanying archive.

10. Viva Voce Preparation

Q. What is a primary key?
A column that uniquely identifies each row. task_id is the primary key.

Q. What does AUTO_INCREMENT do?
MySQL assigns a new task ID automatically.

Q. What is Tkinter?
Python's standard library for building graphical user interfaces.

Q. What does INSERT do?
It adds a new record to a table.

Q. What does UPDATE do in this project?
It changes a task's status to Completed.

Q. Why use a parameterized query?
It passes values separately from the SQL command and handles data safely.

Q. What does commit do?
It saves database changes made by INSERT, UPDATE, or DELETE.

Appendix A. Complete Source Code

The following listings reproduce the project source files included in the ZIP archive. The only user-specific values to edit before execution are the MySQL account settings in db_config.py.

Appendix A1. task_manager/app.py

```
001  import tkinter as tk
002  from tkinter import messagebox, ttk
003
004  import database
005
006
007  class TaskManagerApp:
008      def __init__(self, root):
009          self.root = root
010          self.root.title("Task Management System")
011          self.root.geometry("930x590")
012          self.root.minsize(720, 460)
013          self.selected_task_id = None
014          self.setup_style()
015          self.build_screen()
016          self.refresh_tasks()
017
018      def setup_style(self):
019          style = ttk.Style()
020          style.theme_use("clam")
021          style.configure("TFrame", background="#f4f6fb")
022          style.configure("TLabel", background="#f4f6fb", foreground="#26324a")
023          style.configure("Title.TLabel", font=("Arial", 22, "bold"), foreground="#172554")
024          style.configure("Hint.TLabel", font=("Arial", 10), foreground="#64748b")
025          style.configure("TButton", font=("Arial", 10), padding=(12, 8))
026          style.configure("Accent.TButton", background="#315efb", foreground="white")
027          style.map("Accent.TButton", background=[("active", "#2448c9")])
028          style.configure("Treeview", rowheight=32, font=("Arial", 10))
029          style.configure("Treeview.Heading", font=("Arial", 10, "bold"))
030
031      def build_screen(self):
032          page = ttk.Frame(self.root, padding=24)
033          page.pack(fill="both", expand=True)
034
035          ttk.Label(page, text="My Tasks", style="Title.TLabel").pack(anchor="w")
036          ttk.Label(
037              page,
038              text="Add tasks, track due dates, and mark finished work.",
039              style="Hint.TLabel",
040          ).pack(anchor="w", pady=(4, 18))
041
042          form = ttk.Frame(page)
043          form.pack(fill="x", pady=(0, 16))
044          form.columnconfigure(1, weight=1)
045
046          ttk.Label(form, text="Task title").grid(row=0, column=0, sticky="w", padx=(0, 10), pady=5)
047          self.title_entry = ttk.Entry(form)
048          self.title_entry.grid(row=0, column=1, sticky="ew", pady=5)
049
050          ttk.Label(form, text="Description").grid(row=1, column=0, sticky="w", padx=(0, 10), pady=5)
051          self.description_entry = ttk.Entry(form)
052          self.description_entry.grid(row=1, column=1, sticky="ew", pady=5)
053
054          ttk.Label(form, text="Due date").grid(row=2, column=0, sticky="w", padx=(0, 10), pady=5)
055          self.date_entry = ttk.Entry(form)
056          self.date_entry.grid(row=2, column=1, sticky="ew", pady=5)
057          self.date_entry.insert(0, "YYYY-MM-DD")
058
059          ttk.Label(form, text="Priority").grid(row=3, column=0, sticky="w", padx=(0, 10), pady=5)
060          self.priority_choice = ttk.Combobox(form, values=("Low", "Normal", "High"), state="readonly")
061          self.priority_choice.grid(row=3, column=1, sticky="w", pady=5)
062          self.priority_choice.set("Normal")
063
064          self.add_button = ttk.Button(
065              form, text="Add task", style="Accent.TButton", command=self.add_task
066          )
067          self.add_button.grid(row=0, column=2, rowspan=4, padx=(14, 0), sticky="ns")
068
069          filters = ttk.Frame(page)
070          filters.pack(fill="x", pady=(0, 8))
071          ttk.Label(filters, text="Search").pack(side="left")
072          self.search_entry = ttk.Entry(filters, width=28)
```

```
073         self.search_entry.pack(side="left", padx=8)
074         self.search_entry.bind("<KeyRelease>", lambda event: self.refresh_tasks())
075         ttk.Label(filters, text="Status").pack(side="left", padx=(12, 4))
076         self.status_choice = ttk.Combobox(filters, values=("All", "Pending", "Completed"),
    state="readonly", width=12)
077         self.status_choice.set("All")
078         self.status_choice.pack(side="left")
079         self.status_choice.bind("<<ComboboxSelected>>", lambda event: self.refresh_tasks())
080
081         table_frame = ttk.Frame(page)
082         table_frame.pack(fill="both", expand=True)
083
084         columns = ("id", "title", "description", "due_date", "priority", "status")
085         self.task_table = ttk.Treeview(
086             table_frame, columns=columns, show="headings", selectmode="browse"
087         )
088         self.task_table.heading("id", text="ID")
089         self.task_table.heading("title", text="TASK")
090         self.task_table.heading("description", text="DESCRIPTION")
091         self.task_table.heading("due_date", text="DUE DATE")
092         self.task_table.heading("priority", text="PRIORITY")
093         self.task_table.heading("status", text="STATUS")
094         self.task_table.column("id", width=50, anchor="center", stretch=False)
095         self.task_table.column("title", width=180)
096         self.task_table.column("description", width=260)
097         self.task_table.column("due_date", width=120, anchor="center")
098         self.task_table.column("priority", width=90, anchor="center")
099         self.task_table.column("status", width=110, anchor="center")
100         self.task_table.pack(side="left", fill="both", expand=True)
101         self.task_table.bind("<<TreeviewSelect>>", self.select_task)
102
103         scrollbar = ttk.Scrollbar(
104             table_frame, orient="vertical", command=self.task_table.yview
105         )
106         scrollbar.pack(side="right", fill="y")
107         self.task_table.configure(yscrollcommand=scrollbar.set)
108
109         buttons = ttk.Frame(page)
110         buttons.pack(fill="x", pady=(14, 0))
111         ttk.Button(
112             buttons, text="Mark completed", command=self.mark_completed
113         ).pack(side="left")
114         ttk.Button(buttons, text="Delete task", command=self.remove_task).pack(
115             side="left", padx=8
116         )
117         ttk.Button(buttons, text="Refresh", command=self.refresh_tasks).pack(side="right")
118
119     def refresh_tasks(self):
120         try:
121             tasks = database.get_tasks(self.search_entry.get().strip(), self.status_choice.get())
122             for item in self.task_table.get_children():
123                 self.task_table.delete(item)
124             for task_id, title, description, due_date, priority, status in tasks:
125                 self.task_table.insert(
126                     "",
127                     "end",
128                     values=(task_id, title, description or "", due_date or "", priority, status),
129                 )
130             self.selected_task_id = None
131         except Exception as error:
132             messagebox.showerror("Database error", str(error))
133
134     def select_task(self, event=None):
135         selection = self.task_table.selection()
136         if selection:
137             self.selected_task_id = int(self.task_table.item(selection[0], "values")[0])
138         else:
139             self.selected_task_id = None
140
141     def add_task(self):
142         title = self.title_entry.get().strip()
143         description = self.description_entry.get().strip() or None
144         due_date = self.date_entry.get().strip()
145         if due_date == "YYYY-MM-DD":
146             due_date = ""
```

```
147
148         if not title:
149             messagebox.showwarning("Missing title", "Please enter a task title.")
150             return
151
152         try:
153             database.add_task(title, description, due_date or None, self.priority_choice.get())
154             self.title_entry.delete(0, tk.END)
155             self.description_entry.delete(0, tk.END)
156             self.priority_choice.set("Normal")
157             self.date_entry.delete(0, tk.END)
158             self.date_entry.insert(0, "YYYY-MM-DD")
159             self.refresh_tasks()
160         except Exception as error:
161             messagebox.showerror("Could not add task", str(error))
162
163     def mark_completed(self):
164         if self.selected_task_id is None:
165             messagebox.showinfo("Select a task", "Select a task from the list first.")
166             return
167         try:
168             database.complete_task(self.selected_task_id)
169             self.refresh_tasks()
170         except Exception as error:
171             messagebox.showerror("Could not update task", str(error))
172
173     def remove_task(self):
174         if self.selected_task_id is None:
175             messagebox.showinfo("Select a task", "Select a task from the list first.")
176             return
177         if not messagebox.askyesno("Delete task", "Delete the selected task?"):
178             return
179         try:
180             database.delete_task(self.selected_task_id)
181             self.refresh_tasks()
182         except Exception as error:
183             messagebox.showerror("Could not delete task", str(error))
184
185
186 def main():
187     try:
188         database.create_table()
189     except Exception as error:
190         root = tk.Tk()
191         root.withdraw()
192         messagebox.showerror(
193             "MySQL connection failed",
194             "Check that MySQL is running, the database exists, and task_manager/db_config.py has the
    correct login.\n\n"
195             + str(error),
196         )
197         root.destroy()
198         return
199
200     root = tk.Tk()
201     TaskManagerApp(root)
202     root.mainloop()
203
204
205 if __name__ == "__main__":
206     main()
```

Appendix A2. task_manager/database.py

```
001  import mysql.connector
002
003  from db_config import DB_CONFIG
004
005
006  def connect():
007      return mysql.connector.connect(**DB_CONFIG)
008
009
010  def create_table():
011      connection = connect()
012      cursor = connection.cursor()
013      cursor.execute(
014          """CREATE TABLE IF NOT EXISTS tasks (
015              task_id INT AUTO_INCREMENT PRIMARY KEY,
016              title VARCHAR(100) NOT NULL,
017              description VARCHAR(255),
018              due_date DATE,
019              priority VARCHAR(10) NOT NULL DEFAULT 'Normal',
020              status VARCHAR(15) NOT NULL DEFAULT 'Pending'
021          )"""
022      )
023      cursor.execute("SHOW COLUMNS FROM tasks LIKE 'priority'")
024      if not cursor.fetchone():
025          cursor.execute("ALTER TABLE tasks ADD priority VARCHAR(10) NOT NULL DEFAULT 'Normal'")
026      connection.commit()
027      cursor.close()
028      connection.close()
029
030
031  def get_tasks(search_text="", status="All"):
032      connection = connect()
033      cursor = connection.cursor()
034      query = "SELECT task_id, title, description, due_date, priority, status FROM tasks WHERE (title
     LIKE %s OR description LIKE %s)"
035      values = [f"%{search_text}%", f"%{search_text}%"]
036      if status != "All":
037          query += " AND status = %s"
038          values.append(status)
039      query += " ORDER BY CASE priority WHEN 'High' THEN 1 WHEN 'Normal' THEN 2 ELSE 3 END, due_date,
     task_id"
040      cursor.execute(query, tuple(values))
041      tasks = cursor.fetchall()
042      cursor.close()
043      connection.close()
044      return tasks
045
046
047  def add_task(title, description, due_date, priority):
048      connection = connect()
049      cursor = connection.cursor()
050      cursor.execute(
051          "INSERT INTO tasks (title, description, due_date, priority) VALUES (%s, %s, %s, %s)",
052          (title, description, due_date, priority),
053      )
054      connection.commit()
055      cursor.close()
056      connection.close()
057
058
059  def complete_task(task_id):
060      connection = connect()
061      cursor = connection.cursor()
062      cursor.execute("UPDATE tasks SET status = 'Completed' WHERE task_id = %s", (task_id,))
063      connection.commit()
064      changed = cursor.rowcount
065      cursor.close()
066      connection.close()
067      return changed > 0
068
069
070  def delete_task(task_id):
```

```
    connection = connect()
    cursor = connection.cursor()
    cursor.execute("DELETE FROM tasks WHERE task_id = %s", (task_id,))
    connection.commit()
    changed = cursor.rowcount
    cursor.close()
    connection.close()
    return changed > 0
```

Appendix A3. task_manager/db_config.py

```
001  import json
002  from pathlib import Path
003
004  CONFIG_FILE = Path(__file__).with_name("mysql_config.json")
005  DB_CONFIG = {
006      "host": "localhost",
007      "user": "root",
008      "password": "your_mysql_password",
009      "database": "task_manager_db",
010  }
011  if CONFIG_FILE.exists():
012      DB_CONFIG.update(json.loads(CONFIG_FILE.read_text(encoding="utf-8-sig")))
```

Appendix A4. mac.sh

```
#!/bin/bash
set -e
cd "$(dirname "$0")"

if ! command -v python3 >/dev/null 2>&1; then
    echo "Python 3 is required. Install it from https://www.python.org/downloads/macos/ and run this script again."
    exit 1
fi

if ! command -v mysql >/dev/null 2>&1; then
    if command -v brew >/dev/null 2>&1; then
        brew install mysql
    else
        echo "MySQL is required. Install Homebrew from https://brew.sh, then run this script again."
        exit 1
    fi
fi

if command -v brew >/dev/null 2>&1; then
    brew services start mysql >/dev/null
fi

if [ ! -d .venv ]; then
    python3 -m venv .venv
fi
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install -r requirements.txt

printf "MySQL username [root]: "
read -r DB_USER
DB_USER=${DB_USER:-root}
printf "MySQL password (leave blank if none): "
read -r -s DB_PASSWORD
printf "\n"

MYSQL_PWD="$DB_PASSWORD" mysql -u "$DB_USER" < schema.sql
DB_USER="$DB_USER" DB_PASSWORD="$DB_PASSWORD" DB_NAME="task_manager_db" python3 - <<'PYCONFIG'
import json
import os
from pathlib import Path

config = {"host": "localhost", "user": os.environ["DB_USER"], "password": os.environ["DB_PASSWORD"], "database": os.environ["DB_NAME"]}
Path("task_manager/mysql_config.json").write_text(json.dumps(config, indent=2), encoding="utf-8")
PYCONFIG

.venv/bin/python "task_manager/app.py"
```

Appendix A5. windows.bat

```
@echo off
setlocal EnableExtensions DisableDelayedExpansion
cd /d "%~dp0"
set "PYTHONHOME="
set "PYTHONPATH="
set "TASK_SETUP_LOG=%CD%\setup-diagnostic.txt"
set "TASK_VENV_PYTHON=%CD%\.venv\Scripts\python.exe"
> "%TASK_SETUP_LOG%" echo Task Management System - Python startup diagnostic

if not exist "task_manager\app.py" goto extract_project
if not exist "requirements.txt" goto extract_project
if not exist "schema.sql" goto extract_project

call :find_python
if defined TASK_PYTHON_EXE goto python_ready
where winget >nul 2>nul
if errorlevel 1 goto python_repair_required
echo Installing the full Python 3.13 distribution...
winget install --id Python.Python.3.13 --exact --source winget --scope user --accept-package-agreements --accept-source-agreements
call :find_python
if not defined TASK_PYTHON_EXE goto python_repair_required

:python_ready
echo Using Python: "%TASK_PYTHON_EXE%"
"%TASK_PYTHON_EXE%" -E -c "import encodings, sys; print(sys.version); print('Python:', sys.executable); print('Standard library:', encodings.__file__)" >> "%TASK_SETUP_LOG%" 2>&1
if not exist "%TASK_VENV_PYTHON%" goto rebuild_environment
"%TASK_VENV_PYTHON%" -E -c "import encodings, pip, sys; from pathlib import Path; assert sys.prefix != sys.base_prefix; assert Path(sys.prefix).resolve() == Path('.venv').resolve()" >> "%TASK_SETUP_LOG%" 2>&1
if not errorlevel 1 goto environment_ready

:rebuild_environment
if not exist ".venv" goto create_environment
set "TASK_BACKUP_NAME=.venv-broken-%RANDOM%-%RANDOM%"
if exist "%TASK_BACKUP_NAME%" goto rebuild_environment
echo Rebuilding the project Python environment. The previous folder will be kept as "%TASK_BACKUP_NAME%".
ren ".venv" "%TASK_BACKUP_NAME%"
if errorlevel 1 goto environment_failed

:create_environment
"%TASK_PYTHON_EXE%" -E -m venv ".venv"
if errorlevel 1 goto environment_failed
"%TASK_VENV_PYTHON%" -E -c "import encodings, pip, sys; from pathlib import Path; assert sys.prefix != sys.base_prefix; assert Path(sys.prefix).resolve() == Path('.venv').resolve()" >> "%TASK_SETUP_LOG%" 2>&1
if errorlevel 1 goto environment_failed

:environment_ready
"%TASK_VENV_PYTHON%" -E -m pip install -r "requirements.txt"
if errorlevel 1 goto dependencies_failed

call :find_mysql
if defined TASK_MYSQL_EXE goto mysql_ready
where winget >nul 2>nul
if errorlevel 1 goto mysql_missing
echo Installing MySQL Server. Follow any installer prompts to configure the server and its password.
winget install --id Oracle.MySQL --exact --accept-package-agreements --accept-source-agreements
call :find_mysql
if not defined TASK_MYSQL_EXE goto mysql_missing

:mysql_ready
set "DB_USER="
set "DB_PASSWORD="
set /p "DB_USER=MySQL username [root]: "
if not defined DB_USER set "DB_USER=root"
set /p "DB_PASSWORD=MySQL password - press Enter if blank: "
set "MYSQL_PWD=%DB_PASSWORD%"
"%TASK_MYSQL_EXE%" -u "%DB_USER%" < "schema.sql"
if errorlevel 1 goto database_failed
```

```
066  set "MYSQL_PWD="
067  "%TASK_VENV_PYTHON%" -E -c "import json, os; from pathlib import Path; Path('task_manager/
     mysql_config.json').write_text(json.dumps({'host':'localhost','user':os.environ['DB_USER'],
     'password':os.environ.get('DB_PASSWORD',''),'database':'task_manager_db'}, indent=2), encoding='utf-
     8')"
068  if errorlevel 1 goto settings_failed
069  set "DB_PASSWORD="
070  "%TASK_VENV_PYTHON%" -E "task_manager\app.py"
071  if errorlevel 1 goto application_failed
072  exit /b 0
073
074  :find_python
075  set "TASK_PYTHON_EXE="
076  where py >nul 2>nul
077  if not errorlevel 1 (
078      for %%V in (3 3.14 3.13 3.12 3.11 3.10 3.9) do call :probe_launcher %%V
079  )
080  for /f "delims=" %%P in ('where python 2^>nul') do call :probe_python "%%P"
081  for /f "delims=" %%P in ('where python3 2^>nul') do call :probe_python "%%P"
082  for /d %%D in ("%LocalAppData%\Programs\Python\Python*" "%ProgramFiles%\Python*") do call :
     probe_python "%%~D\python.exe"
083  exit /b 0
084
085  :probe_launcher
086  if defined TASK_PYTHON_EXE exit /b 0
087  for /f "delims=" %%P in ('py -%1 -E -c "import encodings, sys, venv; print(sys.executable)" 2^>nul')
     do call :probe_python "%%P"
088  exit /b 0
089
090  :probe_python
091  if defined TASK_PYTHON_EXE exit /b 0
092  if not exist "%~1" exit /b 0
093  set "TASK_CANDIDATE=%~1"
094  if not "%TASK_CANDIDATE:WindowsApps=%"=="%TASK_CANDIDATE%" exit /b 0
095  >> "%TASK_SETUP_LOG%" echo Checking "%~1"
096  "%~1" -E -c "import encodings, sys, tkinter, venv; assert sys.version_info >= (3,9); assert sys.
     prefix == sys.base_prefix" >> "%TASK_SETUP_LOG%" 2>&1
097  if not errorlevel 1 set "TASK_PYTHON_EXE=%~1"
098  exit /b 0
099
100  :find_mysql
101  set "TASK_MYSQL_EXE="
102  for /f "delims=" %%M in ('where mysql 2^>nul') do set "TASK_MYSQL_EXE=%%M"
103  if defined TASK_MYSQL_EXE exit /b 0
104  for /d %%D in ("%ProgramFiles%\MySQL\MySQL Server *") do if exist "%%~D\bin\mysql.exe" set
     "TASK_MYSQL_EXE=%%~D\bin\mysql.exe"
105  exit /b 0
106
107  :extract_project
108  echo Extract the entire ZIP first, then run windows.bat inside the extracted folder.
109  goto failed
110
111  :python_repair_required
112  echo Windows could not find a working full Python installation with encodings and Tkinter.
113  echo Install Python from https://www.python.org/downloads/windows/ with pip and Tcl/Tk enabled.
114  echo If Python is already installed, open its installer, choose Modify, then Repair.
115  echo Close this window and run windows.bat again after the repair.
116  goto failed
117
118  :environment_failed
119  echo The project Python environment could not be created. Repair the full Python installation and
     try again.
120  goto failed
121
122  :dependencies_failed
123  echo The Python package installation failed. Check the error above and your internet connection,
     then try again.
124  goto failed
125
126  :mysql_missing
127  echo Install and configure MySQL Server from https://dev.mysql.com/downloads/installer/ then run
     this file again.
128  goto failed
129
130  :database_failed
```

```
131 set "MYSQL_PWD="
132 set "DB_PASSWORD="
133 echo MySQL setup failed. Check that MySQL Server is running and your login is correct.
134 goto failed
135
136 :settings_failed
137 set "DB_PASSWORD="
138 echo Could not save the local database settings. Check that the project folder is writable.
139 goto failed
140
141 :application_failed
142 echo The application stopped. Read the error above.
143
144 :failed
145 echo Python startup details are saved in "%TASK_SETUP_LOG%". No MySQL password is included in that
    log.
146 pause
147 exit /b 1
```

Appendix A6. requirements.txt

```
001  mysql-connector-python
```

Appendix B. SQL Setup Script

```
CREATE DATABASE IF NOT EXISTS task_manager_db;
USE task_manager_db;

CREATE TABLE IF NOT EXISTS tasks (
    task_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    description VARCHAR(255),
    due_date DATE,
    priority VARCHAR(10) NOT NULL DEFAULT 'Normal',
    status VARCHAR(15) NOT NULL DEFAULT 'Pending'
);
```

This file is part of the same project archive and should be run once before launching the Python program.